

1. CONTAINS DUPLICATE - BRUTE FORCE

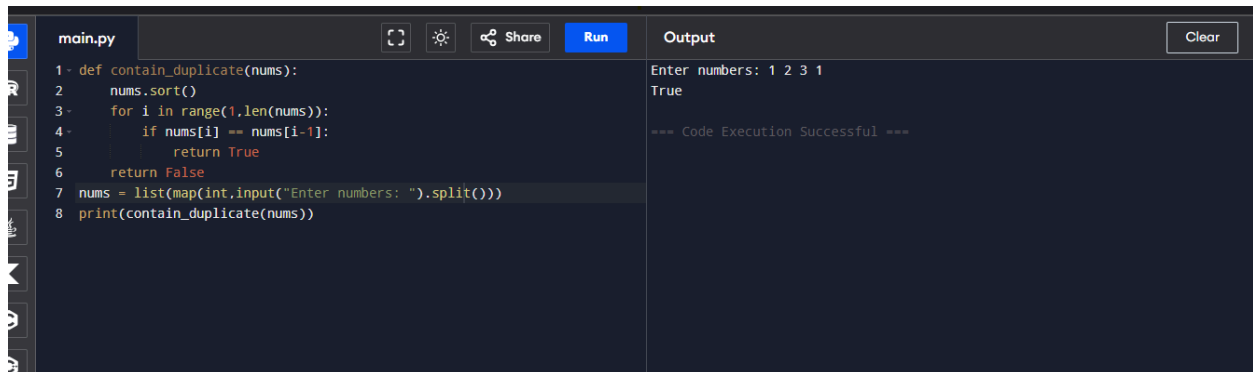


The screenshot shows a code editor with a file named `main.py`. The code implements a brute force algorithm to check for duplicates in an array. It uses two nested loops to compare every element with every other element. The output shows the input "Enter numbers: 1 2 3 3" and the result "True", indicating that there is a duplicate.

```
1- def duplicates(nums):
2-     n = len(nums)
3-     for i in range(n):
4-         for j in range(i+1,n):
5-             if nums[i] == nums[j]:
6-                 return True
7-     return False
8- nums = (list(map(int , input("Enter numbers: ").split()))
9- print(duplicates(nums))
10
```

Output: Enter numbers: 1 2 3 3
True
=== Code Execution Successful ===

2. SORTING METHOD



The screenshot shows a code editor with a file named `main.py`. The code implements a sorting method to check for duplicates. It sorts the array and then iterates through it, checking if any adjacent elements are equal. The output shows the input "Enter numbers: 1 2 3 1" and the result "True", indicating that there is a duplicate.

```
1- def contain_duplicate(nums):
2-     nums.sort()
3-     for i in range(1,len(nums)):
4-         if nums[i] == nums[i-1]:
5-             return True
6-     return False
7- nums = list(map(int,input("Enter numbers: ").split()))
8- print(contain_duplicate(nums))
```

Output: Enter numbers: 1 2 3 1
True
=== Code Execution Successful ===

3. HASH SET



The screenshot shows a code editor with a file named `main.py`. The code implements a hash set solution to check for duplicates. It uses a set to store elements as they are iterated through the array. If an element is already in the set, it means there is a duplicate. The output shows the input "Enter numbers: 1 2 1 3" and the result "True", indicating that there is a duplicate.

```
1- def contain_duplicate(nums):
2-     seen = set()
3-     for num in nums:
4-         if num in seen:
5-             return True
6-         seen.add(num)
7-     return False
8- nums = list(map(int,input("Enter numbers: ").split()))
9- print(contain_duplicate(nums))
```

Output: Enter numbers: 1 2 1 3
True
=== Code Execution Successful ===

4. HASH MAP

```
main.py  [Icons] [Share] [Run] [Clear]
1- def contain_duplicate(nums):
2-     count = {}
3-     for num in nums:
4-         if num in count:
5-             return True
6-         count[num] = 1
7-     return False
8- nums = list(map(int, input("Enter numbers: ").split()))
9- print(contain_duplicate(nums))

Output
Enter numbers: 1 2 3 1
True
=== Code Execution Successful ===
```

5. COMPARE LENGTH SET

```
main.py  [Icons] [Share] [Run] [Clear]
1 def contain_duplicate(nums):
2     return len(nums) != len(set(nums))
3 nums = list(map(int, input("Enter numbers: ").split()))
4 print(contain_duplicate(nums))

Output
Enter numbers: 1 2 1 3
True
=== Code Execution Successful ===
```

FINAL COMPARISON TABLE 🔥

Method	Time Complexity	Space Complexity
Brute Force	$O(n^2)$	$O(1)$
Sorting	$O(n \log n)$	$O(1)$
HashSet	$O(n)$	$O(n)$
Dictionary	$O(n)$	$O(n)$
Set Length Trick	$O(n)$	$O(n)$